

AI ინჟინერიის კურსი PYTHON-ის გამოყენებით

ბლოკი 1: Python Mastery, Logic & Modern Data Flow (28 ლექცია)

ფოკუსი: პროგრამირების ფუნდამენტი, ტიპიზაცია, ობიექტზე ორიენტირებული პროგრამირება და ასინქრონული ლოგიკა.

ლექცია 1: შესავალი (Python და IDE ინსტალაცია)

- რა არის Python
- რა არის IDE
- გარემოს გამართვა

ლექცია 2: მონაცემთა ტიპები და ცვლადები, github-ზე ატვირთვა

- რა არის ცვლადები
- int, float, str, bool
- მონაცემების კონვერტირება ერთი მონაცემთა ტიპიდან მეორეში
- როგორ ატვირთოთ ფაილი Github-ზე

ლექცია 3: სტრინგის მანიპულაციები/მეთოდები, ReGex

- რა არის მეთოდები
- რა მეთოდები გააჩნია სტრინგს
- რატომ არის სტრინგი უცვლელი მონაცემთა ტიპი
- სტრინგის ფორმატირება
- რა არის ReGex

ლექცია 4: პირობითი კონსტრუქციები(if/elif/else), ლოგიკური

ოპერატორები(<,>,<=,>=,==,!=)

- ✗ რა არის პირობითი კონსტრუქციები
- ✗ რა ლოგიკური ოპერატორები არსებობს
- ✗ and და or ლოგიკური ოპერატორები
- ✗ ერთმანეთში ჩადგმული კონსტრუქციები

ლექცია 5: Error Handling

- ✗ რა სახის ერორები არსებობს პითონში
- ✗ try/except/else/finally
- ✗ Exception Hierarchy

ლექცია 6: ციკლები While/For

- ✗ რა არის ციკლი
- ✗ რა სახის ციკლები არსებობს პითონში
- ✗ While ციკლის სინტაქსი და ლოგიკა
- ✗ For ციკლის სინტაქსი და ლოგიკა
- ✗ ერთმანეთში ჩადგმული ციკლები
- ✗ Else (Optional)

ლექცია 7: ლისტები და ტაპლები(List Comprehensions)

- ✗ რა არის კოლექციები
- ✗ რა არის ლისტი
- ✗ რას ნიშნავს დალაგებული კოლექცია(ინდექსები)
- ✗ რა მეთოდები გააჩნია ლისტს
- ✗ რატომ არის ლისტი ცვლადი მონაცემთა ტიპი
- ✗ List comprehensions
- ✗ რა არის ტაპლი
- ✗ ტაპლის unpacking

ლექცია 8: დიქტები და სეტები

- ❌ რა არის დიქტი
- ❌ რა ტიპის ინფორმაცია უნდა იყოს key და value მნიშვნელობებში
- ❌ დიქტის მეთოდები
- ❌ Dict comprehensions
- ❌ რა არის სეტი
- ❌ რას ნიშნავს დაულაგებელი კოლექცია
- ❌ მოქმედებები სიმრავლეებზე
- ❌ frozenset

ლექცია 9: ფუნქციები I

- ❌ რა არის ფუნქციები
- ❌ რატომ გვჭირდება ფუნქციები
- ❌ ფუნქციის აღწერის სინტაქსი
- ❌ ფუნქციის ატრიბუტები(პოზიციური/ქივორდი)
- ❌ Default ატრიბუტები
- ❌ ცვლადი ტიპი, როგორც default ატრიბუტი
- ❌ Return

ლექცია 10: ფუნქციები II

- ❌ რა არის *args და **kwargs
- ❌ რა არის რეკურსიული ფუნქცია
- ❌ რა არის Lambda ფუნქცია
- ❌ ჩაშენებული ფუნქციები(map, filter, reduce, zip, sorted)

ლექცია 11: Type Hinting

- ❌ სტატიკური და დინამიური ტაიპინგი

- ✗ Type hinting პრიმიტიული მონაცემთა ტიპებისთვის(age: int = 25)
- ✗ Type hinting კოლექციებისთვის(scores: list[int] = [1,2,3,4], names: tuple[str, str] = ('John', 'Jane'))
- ✗ Optional ცვლადები
- ✗ Type hinting ფუნქციებისთვის
- ✗ Any
- ✗ MyPy ინსტალაცია და გამოყენება

ლექცია 12: დეკორატორები

- ✗ ერთმანეთში ჩადგმული ფუნქციები
- ✗ LEGB პრინციპი
- ✗ რა არის დეკორატორი
- ✗ ატრიბუტიანი დეკორატორი
- ✗ functools.wraps დეკორატორის გამოყენება

ლექცია 13: გენერატორები

- ✗ რა არის გენერატორი
- ✗ yield ქივორდის ფუნქციონალი
- ✗ ორმხრივი გენერატორი(send(), yield from) - Optional
- ✗ Generator expressions

ლექცია 14: მოდულები, პაკეტები, ვირტუალური გარემო

- ✗ რა არის ვირტუალური გარემო
- ✗ რა არის pip
- ✗ პაკეტების მენეჯმენტის სტრატეგია
- ✗ Requirements.txt გამოყენება

ლექცია 15: ფაილებთან მუშაობა და კონტექსტ მენეჯერი

- ✘ ფაილური სტრუქტურის განხილვა(Windows/Linux)
- ✘ რა სახის ფაილები არსებობს კომპიუტერში
- ✘ ფაილის გახსნა შესაბამისი რეჟიმით(r,w,a,x,r+)
- ✘ რა არის context manager
- ✘ რა არის csv
- ✘ csv მოდულის განხილვა
- ✘ ფაილებისათვის დამახასიათებელი Error Handling

ლექცია 16: შესავალი OOP-ში

- ✘ რა არის OOP და რა პლიუსები გააჩნია
- ✘ Class აღწერა
- ✘ რა არის ობიექტი
- ✘ რა არის ატრიბუტები
- ✘ __init__ კონსტრუქტორი
- ✘ კლასისა და ობიექტის ატრიბუტები

ლექცია 17: OOP პრინციპები

- ✘ ენკაფსულაცია
- ✘ მემკვიდრეობა
- ✘ პოლიმორფიზმი

ლექცია 18: OOP Advanced

- ✘ მრავლობითი მემკვიდრეობა
- ✘ MRO
- ✘ Abstract კლასები
- ✘ Dunder მეთოდები
- ✘ დესკრიპტორები

- ✧ Custom context manager
- ✧ Custom exceptions

ლექცია 19: მონაცემთა სტრუქტურები

- ✧ Big O
- ✧ Linked List
- ✧ Stack
- ✧ Queue

ლექცია 20: სერიალიზაცია და დესერიალიზაცია

- ✧ Json
- ✧ Custom Serialization/Deserialization
- ✧ Dataclass

ლექცია 21: API კომუნიკაცია

- ✧ რა არის API
- ✧ რას ნიშნავს CRUD
- ✧ Requests მოდული
- ✧ Pydantic Schema

ლექცია 22: Multiprocessing & Multithreading

- ✧ Threads
- ✧ GIL
- ✧ Multiprocessing

ლექცია 23: ასინქრონული პროგრამირება

- ✧ რა არის კორუტინა
- ✧ Event Loop
- ✧ async/await
- ✧ ასინქრონული თასქები
- ✧ aiohttp მოდული

ლექცია 24: Git Advanced

- ✘ რა არის Git
- ✘ ძირითადი მოქმედებები
- ✘ Github-ზე მოქმედებები

ლექცია 25: ტესტირება - pytest

- ✘ Fixtures
- ✘ Mocking
- ✘ Parametrization

ლექცია 26: Design Patterns

- ✘ Singleton
- ✘ Factory
- ✘ Observer
- ✘ Strategy

ლექცია 27: კურსის შეჯამება + საუკეთესო პრაქტიკები

- ✘ კურსის შეჯამება
- ✘ რჩევები ფინალური პროექტისთვის
- ✘ Q&A

ლექცია 28: ფინალური პროექტის წარდგენა

ბლოკი 2: AI-Assisted Front-end & API Integration (3 ლექცია)

ფოკუსი: კლიენტ-სერვერის არქიტექტურის გაგება და UI/UX-ის გენერაცია პრომპტ-ინჟინერიით.

- **ლექცია 29: Frontend შესავალი**
 - Html მიმოხილვა
 - Css მიმოხილვა
 - Javascript მიმოხილვა
- **ლექცია 30: Frontend რეალური გამოყენება**
 - Bootstrap მიმოხილვა
 - ☐ ცირე პროექტის მიმოხილვა
- **ლექცია 31: streamlit**
 - ☐ ა არის streamlit
 - ☐ ისთვის გამოყენება
 - Streamlit ინსტალაცია და გაშვება
 - ☐ აგალითების მიმოხილვა

ბლოკი 3: Modern Databases – Relational & Vector Storage (4 ლექცია)

ფოკუსი: მონაცემთა შენახვის არქიტექტურა, PostgreSQL, ვექტორული სივრცეები და ORM-ის გამოყენება.

- **ლექცია 32: რელაციური ბაზების საფუძვლები და PostgreSQL**
 - მონაცემთა ბაზების კონცეფცია და არქიტექტურა.
 - PostgreSQL-ის ინსტალაცია და pgAdmin/DBBeaver-ის მიმოხილვა.
 - ძირითადი SQL ბრძანებები: CRUD ოპერაციები (CREATE, READ, UPDATE, DELETE).
 - ცხრილების შექმნა, მონაცემთა ტიპები და Primary/Foreign გასაღებები (Keys).
- **ლექცია 33: მონაცემთა ბაზების ნორმალიზაცია**
 - ცხრილების დაკავშირება: INNER JOIN, LEFT JOIN, RIGHT JOIN.
 - აგრეგაციები (GROUP BY, COUNT, SUM, AVG) და მონაცემთა გაფილტვრა (HAVING, LIKE, IN).

- ინდექსირების მნიშვნელობა პერფორმანსისთვის.
- **ლექცია 34 : აგრეგირების ფუნქციები**
 - Min/max/sum
- **ლექცია 35: Database in Python (SQLAlchemy)**
 - რა არის ORM და რატომ ვიყენებთ მას?
 - მონაცემთა ბაზასთან ასინქრონული კავშირის დამყარება პითონიდან და სესიების (Sessions) მართვა.

ბლოკი 4: FastAPI Fundamentals & DB Integration (6 ლექცია)

ფოკუსი: API არქიტექტურა, მონაცემთა ვალიდაცია Pydantic-ით და CRUD ოპერაციები ბაზებთან.

- **ლექცია 36: Introduction to APIs & FastAPI**
 - რა არის RESTful API? (Client-Server არქიტექტურა).
 - პირველი FastAPI აპლიკაციის გაშვება Uvicorn-ით.
 - ავტომატური დოკუმენტაცია: Swagger UI დოკუმენტაციის წაკითხვა და ტესტირება.
- **ლექცია 37: Routing, Path & Query Parameters**
 - ენდფოინტების შექმნა სტანდარტული HTTP მეთოდებით (GET, POST, PUT, DELETE).
 - დინამიური URL-ები (Path Parameters) და ფილტრაცია (Query Parameters).
- **ლექცია 38: Data Validation with Pydantic**
 - Pydantic მოდელების შექმნა შემომავალი და გამავალი მონაცემებისთვის (Schemas).
 - მკაცრი ტიპიზაცია და Custom ვალიდატორების დაწერა.
- **ლექცია 39: SQLAlchemy & FastAPI Integration**
 - მე-3 ბლოკში ნასწავლი PostgreSQL-ისა და SQLAlchemy-ის დაკავშირება FastAPI-სთან.
 - Database Session-ის შექმნა Dependency Injection-ის გამოყენებით.
 - **Dependency Injection (DI) in FastAPI**
 - DI-ის კონცეფცია და რატომ არის ის FastAPI-ის "სუპერძალა".
 - კოდის რეუსაბელურობა (მაგ: ბაზასთან კავშირის Session-ის გადაწოდება ფუნქციებისთვის).
 -
- **ლექცია 40: Building CRUD Operations**
 - სრულფასოვანი CRUD ლოგიკის აწყობა.

- პრაქტიკა: სასწავლო კურსების (Courses) და სტუდენტების (Students) ცხრილების შექმნა, წაკითხვა, განახლება და წაშლა.
- **ლექცია 41: Project Structuring & APIRouter**
 - კოდის ორგანიზება დიდ პროექტებში (როგორ ავირიდოთ თავიდან ერთი უზარმაზარი main.py ფაილი).
 - APIRouter-ის გამოყენება მოდულური ენდფოინტებისთვის (მაგ: ცალკე /users და ცალკე /courses როუტერები).

ბლოკი 5: Advanced FastAPI, Security & AI Workloads (8 ლექცია)

ფოკუსი: ავთენტიფიკაცია, როლების მართვა და ასინქრონული პროცესები, რომლებიც კრიტიკულია AI აპლიკაციებისთვის.

- **ლექცია 42: Authentication & Security (JWT)**
 - უსაფრთხოების საფუძვლები, პაროლების ჰეშირება (Passlib, Bcrypt).
 - JSON Web Tokens (JWT) შექმნა და ვალიდაცია.
 - ენდფოინტების დაცვა Depends() გამოყენებით.
- **ლექცია 43: Role-Based Access Control (RBAC)**
 - ავტორიზაცია სისტემაში: როგორ გავმიჯნოთ მომხმარებლები.
 - პრაქტიკა: როლების (მაგ: Course Creator, Expert, Student) ლოგიკის იმპლემენტაცია. (მაგ: მხოლოდ "Expert" როლს შეუძლია რთული პროექტების Peer-review ენდფოინტზე წვდომა).
- **ლექცია 44: Middleware & CORS**
 - რა არის Cross-Origin Resource Sharing (CORS) და რატომ გვჭირდება ის, როცა Front-end (მაგ. ჩვენი Streamlit ან მომავალში Angular/React) სხვა პორტზეა.
 - Custom Middleware-ების დაწერა (მაგ: request-ების დროის დამთვლელი ლოგერი).
- **ლექცია 45: Background Tasks & Async Workloads**
 - რატომ არ უნდა დავაყოვნოთ მომხმარებელი მძიმე ოპერაციების დროს?
 - BackgroundTasks-ის გამოყენება FastAPI-ში.
 - პრაქტიკა: LLM API-ს გამოძახება დავალებების ავტომატური შეფასებისთვის (Automatic Exercise Grading) ფონურ რეჟიმში, ისე რომ API-მ მაშინვე დააბრუნოს პასუხი "დავალება მიღებულია".
- **ლექცია 46: Real-time Communication: WebSockets I**
 - HTTP-ისა და WebSockets-ის შედარება.
 - სერვერსა და კლიენტს შორის მუდმივი, ორმხრივი კავშირის დამყარება.
- **ლექცია 47: WebSockets II (Streaming AI Responses)**
 - WebSockets-ის პრაქტიკული გამოყენება AI ჩატბოტებისთვის.
 - პრაქტიკა: LLM-ის პასუხის სტრიმინგი რეალურ დროში (როგორც ChatGPT ბეჭდავს ტოკენებს სათითაოდ), Front-end-ად Streamlit-ის ან Postman-ის გამოყენებით.
- **ლექცია 48: Error Handling & Data Serialization**
 - გლობალური Exception Handler-ების დაწერა.

- მონაცემთა სწორი სერიალიზაცია (კომპლექსური ობიექტების, მაგალითად თარიღების და რიცხვების დაბრუნება JSON ფორმატში).
- **ლექცია 49: Block 4 & 5 Mini-Project (Full-Stack API Integration)**
 - **Backend (FastAPI & PostgreSQL):** სრულფასოვანი API-ს აწყობა ავტორიზაციითა და როლების მართვის ლოგიკით (მაგალითად: Course Creator, Expert და Student როლების გამიჯვნა ენდფოინთებზე). მონაცემების შენახვა SQLAlchemy-ის გამოყენებით.
 - **Frontend (Streamlit):** მე-2 ბლოკში ნასწავლი Streamlit-ით ინტერფეისის შექმნა, რომელიც requests მოდულის მეშვეობით დაუკავშირდება ჩვენს FastAPI ენდფოინთებს.
 - **Full-Stack ციკლი:** მომხმარებლის ლოგინის, კურსების სიის გამოტანისა და სისტემაში ახალი მონაცემის დამატების (POST request) ვიზუალიზაცია.
 - **შედეგი:** კლიენტის (Streamlit) და სერვერის (FastAPI) დამოუკიდებელი, მაგრამ სინქრონული მუშაობის პრაქტიკული დემონსტრირება.

ბლოკი 6: Applied AI, RAG & Agentic Systems (12 ლექცია)

ფოკუსი: LLM-ების ინტეგრაცია პროდუქტში, პრომპტ-ინჟინერია დეველოპერებისთვის, RAG არქიტექტურა და დამოუკიდებელი აგენტების შექმნა.

- **ლექცია 50: LLM APIs & Developer Prompting**
 - OpenAI / Anthropic API-ებთან პირდაპირი კომუნიკაცია პითონიდან.
 - System, User და Assistant მესიჯების როლი.
 - Context Window-ს მართვა და ტოკენების ოპტიმიზაცია.
- **ლექცია 51: Structured Outputs (Pydantic + LLM)**
 - რატომ არის კრიტიკული LLM-ისგან სტრუქტურირებული (JSON) პასუხის მიღება API-სთვის.
 - FastAPI-ში უკვე ნასწავლი Pydantic მოდელების გამოყენება LLM-ის პასუხის გასაწვდომად (მაგ: დავაბრუნოთ შეფასება 1-დან 10-მდე და კომენტარი ზუსტი სტრუქტურით).
- **ლექცია 52: LangChain ეკოსისტემის საფუძვლები**
 - Models, Prompts და Output Parsers (LCEL - LangChain Expression Language).
 - ჯაჭვების (Chains) აწყობა და მეხსიერების (Memory) მართვა.
- **ლექცია 53: ვექტორული მონაცემთა ბაზები, Embeddings და Qdrant**
 - რა არის Embedding (ტექსტის რიცხვობრივ მასივად ქცევა)?
 - სემანტიკური ძებნა ტრადიციული Keyword ძებნის წინააღმდეგ.

- PostgreSQL-ში pgvector გაფართოების (Extension) ინსტალაცია და კონფიგურაცია.
- **ლექცია 54: RAG System I – Data Ingestion (pgvector-თან ინტეგრაცია)**
 - Document Loaders (PDF-ების, Markdown ფაილების, კოდის წაკითხვა).
 - ტექსტის ჭკვიანური დაჭრა (RecursiveCharacterTextSplitter).
 - Embeddings გენერაცია და უკვე შექმნილ PostgreSQL (pgvector) ბაზაში შენახვა FastAPI-ის ენდფოინტის გავლით.
- **ლექცია 55: RAG System II – Retrieval & Generation**
 - ვექტორული ძებნა ბაზიდან (Similarity Search).
 - მოძიებული კონტენტის პრომპტში ჩასმა და პასუხის გენერაცია.
- **ლექცია 56: Function Calling & Tools**
 - როგორ მივცეთ LLM-ს "ხელები" (Tools).
 - ფუნქციების აღწერა და მოდელისთვის გადაწოდება (მაგალითად, LLM-ს მივცეთ უფლება, გამოიძახოს FastAPI ენდფოინტი ბაზიდან სტუდენტის ქულების ამოსაღებად).
- **ლექცია 57: Introduction to Agents & LangGraph**
 - რატომ არ არის საკმარისი სტანდარტული ჯაჭვები (Chains) და რა არის Agent?
 - LangGraph-ის კონცეფცია: State (მდგომარეობა), Nodes (კვანძები) და Edges (კავშირები).
- **ლექცია 58: LangGraph Deep Dive – Routing & Cyclic Graphs**
 - პირობითი გადასვლები (Conditional Edges) აგენტის ლოგიკაში.
 - ციკლური გრაფები: როგორ მივცეთ აგენტს უფლება იფიქროს და სცადოს ხელახლა, სანამ სწორ პასუხს არ მიიღებს.
- **ლექცია 59: Multi-Agent Systems & Human-in-the-Loop**
 - რამდენიმე აგენტის კოლაბორაცია (მაგ: ერთი წერს კოდს, მეორე ამოწმებს (Peer-review)).
 - პროცესის შეჩერება ადამიანის (ადმინისტრატორის) დასტურის მოლოდინში.
- **ლექცია 60: Agent as a FastAPI Service**
 - LangGraph-ში აწყობილი აგენტის შეფუთვა FastAPI-ის როუტერში.
 - ენდფოინტის შექმნა (მაგ. POST /api/chat), რომელიც იღებს მომხმარებლის ტექსტს, უშვებს აგენტს და აბრუნებს შედეგს.
- **ლექცია 61: Full-Stack AI: Streamlit + FastAPI + Agents**
 - მე-2 ბლოკში ნასწავლი Streamlit-ის დაკავშირება ჩვენს FastAPI ენდფოინტებთან.
 - აგენტის მოქმედებების (ფიქრის პროცესის) ვიზუალიზაცია მომხმარებლისთვის ფრონტენდზე.
- **ლექცია 62: AI Observability with LangSmith**
 - სისტემის მონიტორინგი.
 - აგენტის ნაბიჯების, დახარჯული ტოკენებისა და შეცდომების (Traces) ლოგირება რეალურ დროში.

ბლოკი 7: Production-Grade Deployment & Final Project (8 ლექცია)

- **ლექცია 63: Secrets Management**
 - პროდაქშენისთვის მომზადება: .env ფაილების მართვა და API გასაღებების უსაფრთხოება.
 - პროექტის სტრუქტურირება დეპლოიშენტისთვის.
- **ლექცია 64: Containerization (Dockerizing Backend & Frontend)**
- **ლექცია 65: Linux Server Provisioning (DigitalOcean)**
- **ლექცია 66: Nginx as a Web Server & Reverse Proxy**
- **ლექცია 67: CI/CD Pipeline (GitHub Actions)**
 - როგორ ავირიდოთ სერვერზე ხელით შესვლა ყოველი ცვლილებისას.
- **ლექცია 68: Final Assembly & CORS** - CORS პოლიტიკის კონფიგურაცია ბექენდზე და კლიენტ-სერვერის სრული არქიტექტურის შეერთება ღრუბელში.
- **ლექცია 69: Final Project** - LMS-ის AI მოდულის ტესტირება რეალურ გარემოში და ლოგების მონიტორინგი.
- **ლექცია 70: პროექტების პრეზენტაცია (Demo Day)**